

COURSE 6: DATABASES AND SQL FOR DATA SCIENCE **WITH PYTHON**

MODULE 1: GETTING STARTED WITH SQL – NOTES

1.1 COURSE INTRODUCTION

We live in a **data-driven world** where huge amounts of data are generated daily. Managing, storing, and analyzing this data is essential for businesses, science, healthcare, banking, and more.

Why Learn SQL?

- SQL is used to **query, update, and manage data**
- Essential skill for **Data Analysts, Data Scientists, Engineers**
- Used globally in **industry and academia**
- Works with Python for data analysis

Course Modules Overview

Week Topic

- | | |
|---|---------------------------------|
| 1 | SQL Basics |
| 2 | Relational Databases & Tables |
| 3 | Intermediate SQL |
| 4 | Accessing Databases with Python |
| 5 | Final Assignment |
| 6 | Advanced SQL |

1. What is SQL?

Definition:

SQL stands for **Structured Query Language**

Used to:

- Retrieve data
- Insert data
- Update data
- Delete data
- Create tables/databases

SQL is mainly used with **Relational Databases**

2. What is Data?

Data = Collection of facts such as:

- Names
- Numbers
- Addresses
- Emails
- Images

Examples:

- Bank records
- Credit card data
- Customer details

3. What is a Database?

Definition:

A **database** is an organized repository that stores data.

It provides functionality to:

- Add data
- Modify data
- Query data

- Secure data

4. What is DBMS?

DBMS = Database Management System

Software used to manage databases.

Examples:

- MySQL
- Oracle
- IBM DB2
- PostgreSQL
- SQL Server

5. What is RDBMS?

RDBMS = Relational Database Management System

Stores data in **tables** with:

- Rows
- Columns

Allows relationships between tables.

Example:

Employees table linked to Departments table.

6. Table Structure

Example: Employee Table

EmpID	FirstName	LastName	City
101	John	Smith	NY

Columns:

Attributes/properties

Rows:

Actual records

7. Five Basic SQL Commands**Command Purpose**

CREATE	Create table
INSERT	Add data
SELECT	Retrieve data
UPDATE	Modify data
DELETE	Remove data

8. SELECT Statement**Most Important SQL Command**

Used to retrieve data from a table.

Basic Syntax**Select Specific Columns**

```
SELECT column1, column2 FROM table_name;
```

Select All Columns

```
SELECT * FROM table_name;
```

Example Table: COUNTRY

ID	Name	CCode
1	United States	US
2	China	CH

ID	Name	CCode
3	Japan	JA
4	Germany	GE
5	India	IN

1.2 EXAMPLE QUERIES

Example 1: Show Entire Table

```
SELECT * FROM COUNTRY;
```

Output:

All rows and columns

Example 2: Show Specific Columns

```
SELECT ID, Name FROM COUNTRY;
```

Output:

Only ID and Name

Example 3: Filter Rows

```
SELECT * FROM COUNTRY WHERE ID <= 5;
```

Output:

Rows where ID is 5 or less

Example 4: Match Exact Value

```
SELECT * FROM COUNTRY WHERE CCode = 'CA';
```

Output:

ID	Name	CCode
9	Canada	CA

1.3 WHERE CLAUSE

Used to filter rows.

Syntax:

SELECT columns FROM table WHERE condition;
--

What is Predicate?

A predicate is a condition that evaluates to:

- TRUE
- FALSE
- UNKNOWN

Example:

ID = 5

Comparison Operators

Operator	Meaning
=	Equal
>	Greater than
<	Less than
>=	Greater than or equal
<=	Less than or equal
<> or !=	Not equal

Lab: Simple SELECT Statements

1.4 SQL EXPRESSIONS: COUNT, DISTINCT, LIMIT

1. COUNT Function

◆ Definition

- COUNT() is an **aggregate function** in SQL.
- It returns the **number of rows** that match a condition.

◆ Syntax

```
SELECT COUNT(*) FROM table_name;
```

◆ Examples

- Count total rows in a table:

```
SELECT COUNT(*) FROM MEDALS;
```

- Count rows matching a condition:

```
SELECT COUNT(COUNTRY)
FROM MEDALS
WHERE COUNTRY = 'CANADA';
```

◆ Key Points

- COUNT(*) → counts all rows
- COUNT(column) → counts non-null values in that column

2. DISTINCT Keyword

◆ Definition

- DISTINCT removes **duplicate values**
- Returns only **unique values**

◆ Syntax

```
SELECT DISTINCT column_name FROM table_name;
```

◆ Example

- Get unique countries that won gold medals:

```
SELECT DISTINCT COUNTRY  
FROM MEDALS  
WHERE MEDALTYPE = 'GOLD';
```

◆ Key Points

- Eliminates repetition
- Useful for finding unique categories, values, or groups

3. LIMIT Clause

◆ Definition

- LIMIT restricts the **number of rows returned**

◆ Syntax

```
SELECT * FROM table_name LIMIT number;
```

◆ Examples

- Get first 10 rows:

```
SELECT * FROM MEDALS LIMIT 10;
```

- Get 5 rows for a specific year:

```
SELECT *  
FROM MEDALS  
WHERE YEAR = 2018  
LIMIT 5;
```

◆ Key Points

- Used to preview data
- Prevents large output
- Very useful for testing queries

Summary Table

Keyword	Purpose	Example Use
COUNT	Counts rows	Total records, filtered counts
DISTINCT	Removes duplicates	Unique countries, values
LIMIT	Restricts output rows	Preview first N rows

Overall Concept

These are **basic SQL tools used with SELECT statements**:

- COUNT → measures size of data
- DISTINCT → cleans duplicates
- LIMIT → controls output size

Lab: COUNT, DISTINCT, LIMIT

1.5 SQL DML STATEMENTS: INSERT, UPDATE, DELETE

1. INSERT Statement

◆ Definition

- INSERT is a **DML (Data Manipulation Language)** statement.
- It is used to **add new rows into a table**.

◆ Syntax

```
INSERT INTO table_name (column1, column2, ...)  
VALUES (value1, value2, ...);
```

◆ Example (Single Row)

```
INSERT INTO AUTHOR (AUTHOR_ID, LASTNAME, FIRSTNAME, EMAIL, CITY,  
COUNTRY)  
VALUES ('A1', 'Chong', 'Raul', 'rfc@ibm.com', 'Toronto', 'CA');
```

◆ Multiple Rows Insert

```
INSERT INTO AUTHOR (AUTHOR_ID, LASTNAME, FIRSTNAME, EMAIL, CITY,
COUNTRY)
VALUES
('A1', 'Chong', 'Raul', 'rfc@ibm.com', 'Toronto', 'CA'),
('A2', 'Ahuja', 'Rav', 'ra@ibm.com', 'New York', 'US');
```

◆ Key Points

- Number of columns = number of values must match
- Can insert **one or multiple rows at once**

2. UPDATE Statement

◆ Definition

- UPDATE modifies existing data in a table.
- It is a **DML statement**.

◆ Syntax

```
UPDATE table_name
SET column1 = value1, column2 = value2
WHERE condition;
```

◆ Example

```
UPDATE AUTHOR
SET FIRSTNAME = 'Lakshmi', LASTNAME = 'Katta'
WHERE AUTHOR_ID = 'A2';
```

◆ Important Rule ⚠

- If **WHERE** clause is missing → **ALL** rows get updated

◆ Key Points

- Used to modify existing records
- Always use WHERE carefully

3. DELETE Statement

◆ Definition

- DELETE removes rows from a table.
- It is also a **DML statement**.

◆ Syntax

```
DELETE FROM table_name  
WHERE condition;
```

◆ Example

```
DELETE FROM AUTHOR  
WHERE AUTHOR_ID IN ('A2', 'A3');
```

◆ Important Rule ⚠

- If **WHERE clause is missing** → **ALL rows are deleted**

◆ Key Points

- Used to remove specific records
- Must use WHERE to avoid deleting full table

🧠 Summary Table

Statement	Purpose	Risk Without WHERE
INSERT	Add new rows	Safe
UPDATE	Modify existing rows	Updates entire table
DELETE	Remove rows	Deletes entire table

Lab: INSERT, UPDATE, and DELETE

MODULE SUMMARY: GETTING STARTED WITH SQL

Key Concepts

1. SQL & Data Manipulation Language (DML)

- SQL is used to **query and manage data in databases**
- DML (Data Manipulation Language) is used to:
 - Read data
 - Modify data

◆ DML Statements:

- SELECT → retrieve data
- INSERT → add new data
- UPDATE → modify existing data
- DELETE → remove data

2. WHERE Clause & Predicate

- WHERE is used to **filter records**
- It uses a **predicate (condition)** that evaluates to TRUE/FALSE

Example:

```
SELECT * FROM table_name WHERE city = 'Rome';
```

3. SELECT Statement Applications

- Used to retrieve data from database tables
- Can select:
 - All columns
 - Specific columns
 - Filtered data

4. SQL Expressions with SELECT

- COUNT → counts rows
- DISTINCT → removes duplicates
- LIMIT → restricts number of rows returned

5. Data Modification Statements

- INSERT → add new rows
- UPDATE → modify existing rows
- DELETE → remove rows

SQL CHEAT SHEET (QUICK REVISION)

◆ 1. SELECT

Purpose:

Retrieve data from a table

Syntax:

```
SELECT column1, column2 FROM table_name;
```

Example:

```
SELECT city FROM placeofinterest;
```

◆ 2. WHERE

Purpose:

Filter records

Syntax:

```
SELECT * FROM table_name WHERE condition;
```

Example:

```
SELECT * FROM placeofinterest WHERE city = 'Rome';
```

◆ 3. COUNT

Purpose:

Count rows (non-null values)

Syntax:

```
SELECT COUNT(column_name) FROM table_name;
```

Example:

```
SELECT COUNT(country)
FROM placeofinterest
WHERE country = 'Canada';
```

◆ 4. DISTINCT

Purpose:

Return unique values

Syntax:

```
SELECT DISTINCT column_name FROM table_name;
```

Example:

```
SELECT DISTINCT country
FROM placeofinterest;
```

◆ 5. LIMIT

Purpose:

Restrict number of rows returned

Syntax:

```
SELECT * FROM table_name LIMIT number;
```

Example:

```
SELECT * FROM placeofinterest LIMIT 5;
```

◆ 6. INSERT

Purpose:

Add new rows

Syntax:

```
INSERT INTO table_name (col1, col2)
VALUES (val1, val2);
```

Example:

```
INSERT INTO placeofinterest (name, type, city, country, airport)
VALUES ('Niagara Waterfalls', 'Nature', 'Toronto', 'Canada', 'Pearson');
```

◆ 7. UPDATE

Purpose:

Modify existing data

Syntax:

```
UPDATE table_name
SET column = value
WHERE condition;
```

Example:

```
UPDATE placeofinterest
SET name = 'Niagara Falls'
WHERE name = 'Niagara Waterfalls';
```

⚠ Without WHERE, ALL rows get updated

◆ 8. DELETE

Purpose:

Remove rows from table

Syntax:

```
DELETE FROM table_name WHERE condition;
```

Example:

```
DELETE FROM placeofinterest  
WHERE city = 'Toronto';
```

⚠ Without WHERE, ALL rows get deleted

MODULE 2: INTRODUCTION TO RELATIONAL DATABASES AND TABLES

2.1 RELATIONAL DATABASE CONCEPTS

◆ What is a Relational Model?

- Most widely used database model.
- Stores data in **tables (rows and columns)**.
- Provides:
 - Logical data independence
 - Physical data independence
 - Storage independence

◆ Entity-Relationship (ER) Model

- Used to **design databases before implementation**.
- Components:
 - **Entity** → Real-world object (e.g., Book, Author)
 - **Attribute** → Properties of entity (e.g., title, name)

◆ Mapping ER → Relational Model

ER Concept	Database Concept
Entity	Table
Attribute	Column
Instance	Row

◆ Primary Key

- Uniquely identifies each row in a table.
- Prevents duplicate records.
- Cannot be NULL.

◆ Foreign Key

- A primary key from another table.
- Used to create relationships between tables.

◆ Data Types

- CHAR → Fixed length text
- VARCHAR → Variable length text
- INT → Integer numbers
- DECIMAL → Decimal numbers
- DATE/TIME → Date and time values

2.2 SQL STATEMENT TYPES

1. DDL (Data Definition Language)

Used to define database structure.

- CREATE
- ALTER
- DROP
- TRUNCATE

2. DML (Data Manipulation Language)

Used to manage data in tables.

- SELECT
 - INSERT
 - UPDATE
 - DELETE
-

2.3 SQL STATEMENTS

1. CREATE TABLE Statement

◆ Purpose

- Used to create new tables in database.

◆ Syntax

```
CREATE TABLE table_name (  
    column1 datatype,  
    column2 datatype,  
    ...  
);
```

◆ Example

```
CREATE TABLE author (  
    author_id CHAR(2) PRIMARY KEY NOT NULL,  
    firstname VARCHAR(15),  
    lastname VARCHAR(15),  
    email VARCHAR(40),  
    city VARCHAR(15),  
    country CHAR(2)  
);
```

2. ALTER TABLE Statement

◆ Purpose

Modify existing table structure.

◆ Operations:

- Add column
- Modify column datatype
- Drop column

◆ Examples:

```
ALTER TABLE author ADD telephone BIGINT;
```

```
ALTER TABLE author MODIFY telephone CHAR(20);
```

```
ALTER TABLE author DROP COLUMN telephone;
```

3. DROP TABLE

◆ Purpose

- Deletes entire table (structure + data)

◆ Syntax:

```
DROP TABLE table_name;
```

4. TRUNCATE TABLE

◆ Purpose

- Deletes all rows in a table
- Keeps table structure intact

◆ Syntax:

```
TRUNCATE TABLE table_name;
```

5. SELECT Statement (Data Retrieval)

◆ Basic Syntax:

```
SELECT column1, column2 FROM table;
```

◆ Retrieve all columns:

```
SELECT * FROM table;
```

◆ Filtering with WHERE:

```
SELECT * FROM table WHERE condition;
```

6. COUNT, DISTINCT, LIMIT

◆ COUNT

- Counts number of rows

```
SELECT COUNT(*) FROM table;
```

◆ DISTINCT

- Removes duplicates

```
SELECT DISTINCT country FROM table;
```

◆ LIMIT

- Restricts number of rows returned

```
SELECT * FROM table LIMIT 5;
```

7. INSERT Statement

◆ Purpose

Adds new rows to table.

◆ Syntax:

```
INSERT INTO table_name (col1, col2)  
VALUES (value1, value2);
```

◆ **Multiple rows:**

```
INSERT INTO table_name VALUES  
(value1, value2),  
(value3, value4);
```

8. UPDATE Statement

◆ **Purpose**

Modifies existing data.

◆ **Syntax:**

```
UPDATE table_name  
SET column = value  
WHERE condition;
```

⚠ Without WHERE → updates ALL rows.

9. DELETE Statement

◆ **Purpose**

Removes rows from table.

◆ **Syntax:**

```
DELETE FROM table_name  
WHERE condition;
```

⚠ Without WHERE → deletes ALL rows.

LAB : CREATE, ALTER, TRUNCATE, DROP

2.4 UNDERSTANDING RELATIONAL MODEL CONSTRAINTS

1. 🗝️ Relational Model Constraints

Constraints ensure **data accuracy, consistency, and reliability** in a database.

A. Entity Integrity Constraint

◆ Definition:

Ensures every table has a **Primary Key** that uniquely identifies each row.

◆ Rules:

- Must be **unique**
- Cannot be **NULL**
- Each row must be identifiable

◆ Example:

```
CREATE TABLE BookShop (  
    BOOK_ID INT PRIMARY KEY,  
    TITLE VARCHAR(100),  
    AUTHOR_ID INT  
);
```

◆ Key Idea:

👉 No duplicate or missing primary key values allowed

B. Referential Integrity Constraint

◆ Definition:

Ensures **foreign key values match primary key values in another table**.

◆ Purpose:

Maintains valid relationships between tables.

◆ Example:

```
CREATE TABLE BookShop_AuthorDetails (AUTHOR_ID INT PRIMARY KEY,  
    AUTHOR_NAME VARCHAR(100));  
CREATE TABLE BookShop (BOOK_ID INT PRIMARY KEY, TITLE
```

```
VARCHAR(100),AUTHOR_ID INT,FOREIGN KEY (AUTHOR_ID)REFERENCES  
BookShop_AuthorDetails(AUTHOR_ID));
```

◆ **Key Idea:**

👉 Prevents “orphan records” (invalid references)

C. Domain Integrity Constraint

◆ **Definition:**

Ensures values in a column are **valid according to rules (datatype, format, range, etc.)**

◆ **Rules:**

- Correct data type
- Valid range
- NOT NULL (if required)
- CHECK conditions

◆ **Example:**

```
CREATE TABLE BookShop (  
    BOOK_ID INT PRIMARY KEY,  
    TITLE VARCHAR(100) NOT NULL,  
    PRICE DECIMAL(5,2) CHECK (PRICE >= 0),  
    PUBLISHED_DATE DATE  
);
```

◆ **Key Constraints:**

✓ **NOT NULL**

- Column cannot be empty

✓ **CHECK**

- Ensures valid values (e.g., price ≥ 0)

2. 🧠 Summary of Constraints

Constraint Type	Purpose
Entity Integrity	Ensures unique primary key
Referential Integrity	Maintains valid relationships
Domain Integrity	Ensures valid column values

2.5 📄 SQL SCRIPTS

◆ What are SQL Scripts?

- A file containing multiple SQL commands
- Usually saved as .sql
- Executed sequentially

◆ Uses of SQL Scripts

✓ 1. Create Tables

- Build database structure

✓ 2. Drop Tables

- Remove existing tables safely

✓ 3. Insert Data

- Populate tables

✓ 4. Update Data

- Modify existing records

✓ 5. Delete Data

- Remove unwanted records

✓ 6. Create Views

- Virtual tables for simplified queries

✓ 7. Stored Procedures

- Predefined SQL logic

✓ 8. Triggers

- Auto-executed actions on events (INSERT/UPDATE/DELETE)

2.6 EXAMPLE SQL SCRIPT STRUCTURE

```
DROP TABLE IF EXISTS PATIENTS;

CREATE TABLE PATIENTS (
    PATIENT_ID CHAR(9) PRIMARY KEY,
    FIRST_NAME VARCHAR(15),
    LAST_NAME VARCHAR(15)
);
```

◆ Important Notes:

- SQL scripts run **line by line (sequentially)**
- Each statement ends with ;
- Always use DROP TABLE IF EXISTS before CREATE (best practice)

[LAB: CREATE AND LOAD TABLES USING SQL SCRIPTS](#)

2.7 HOW TO CREATE A DATABASE INSTANCE ON CLOUD (IBM DB2 ON CLOUD)

2.7.1 What is a Cloud Database?

A **Cloud database** is a database service that runs on a cloud platform and is accessed over the internet.

◆ Key Idea:

Instead of installing database software on your system, you use it online.

2.7.2 🌟 Advantages of Cloud Databases

✓ 1. Easy Access

- Accessible from anywhere using browser or APIs

✓ 2. Scalability

- Storage and computing power can increase/decrease automatically

✓ 3. Cost Efficient

- Pay only for what you use

✓ 4. Backup & Disaster Recovery

- Data is stored in multiple locations for safety

2.7.3 🍌 Examples of Cloud Databases

- IBM Db2 on Cloud
- Amazon Web Services Amazon RDS
- Microsoft Azure Azure SQL Database
- Oracle Oracle Cloud Database
- PostgreSQL on Cloud platforms

2.7.4 🧠 What is a Database Instance?

A **database instance (DBaaS)** is a ready-to-use database service in the cloud.

◆ It provides:

- Database storage
- Tables
- SQL execution environment
- No need for hardware/software setup

2.7.5 How Cloud Database Works

1. Create database instance in cloud
2. Store data in tables
3. Connect using application or web console
4. Send SQL queries
5. Receive results from database

2.7.6 Steps to Create IBM Db2 on Cloud Instance

Step 1: Open IBM Cloud

- Go to IBM Cloud catalog

Step 2: Select Db2 Service

- Choose **Db2 on Cloud**
- Select **Lite (Free) plan**

Step 3: Configure Instance

Fill details like:

- Service name
- Region (location)
- Resource group

Step 4: Create Service

- Click **Create**
- Instance will be provisioned

Step 5: Open Dashboard

- Go to IBM Cloud dashboard
- Open your Db2 service

Step 6: Open Console

- Click **“Open Console”**
- You can now:

- Create tables
- Run SQL queries
- Load data

Step 7: Create Credentials

You must generate **service credentials**.

These include:

- Username
- Password
- Host name
- Port number
- Database name

Step 8: Connect Applications

Use credentials to:

- Connect Python / apps
- Run SQL queries remotely

MODULE 3 NOTES: INTERMEDIATE SQL

3.1 USING STRING PATTERNS

◆ 1. Using LIKE

✓ Purpose:

Used when you **don't know exact values** but want to search using patterns.

✓ Syntax:

```
SELECT column_name  
FROM table_name  
WHERE column_name LIKE 'pattern';
```

✓ Wildcards:

- % → Represents **zero or more characters**
- _ → Represents **single character** (not mentioned above but useful)

✓ Examples:

-- Names starting with 'R'

```
SELECT firstname FROM author WHERE firstname LIKE 'R%';
```

-- Names ending with 'n'

```
SELECT firstname FROM author WHERE firstname LIKE '%n';
```

-- Names containing 'ar'

```
SELECT firstname FROM author WHERE firstname LIKE '%ar%';
```

◆ 2. Using Ranges (BETWEEN)

✓ Purpose:

Filter values within a **range (inclusive)**

✓ Syntax:

```
SELECT column_name  
FROM table_name  
WHERE column_name BETWEEN value1 AND value2;
```

✓ Example:

```
SELECT * FROM book  
WHERE pages BETWEEN 290 AND 300;
```

👉 Equivalent to:

```
WHERE pages >= 290 AND pages <= 300;
```

◆ 3. Using Sets (IN)

✓ Purpose:

Filter using **multiple values without repeating conditions**

✓ **Syntax:**

```
SELECT column_name  
FROM table_name  
WHERE column_name IN (value1, value2, ...);
```

✓ **Example:**

```
SELECT * FROM author  
WHERE country IN ('India', 'Canada', 'China');
```

◆ **4. Sorting Results (ORDER BY)**

✓ **Purpose:**

Sort query results in **ascending or descending order**

✓ **Syntax:**

```
SELECT column_name  
FROM table_name  
ORDER BY column_name [ASC|DESC];
```

✓ **Key Points:**

- Default → **ASC (ascending)**
- Use DESC → descending order

✓ **Examples:**

-- Ascending

```
SELECT title FROM book ORDER BY title;
```

-- Descending

```
SELECT title FROM book ORDER BY title DESC;
```

-- Using column number

```
SELECT title, pages FROM book ORDER BY 2;
```

◆ **5. Eliminating Duplicates (DISTINCT)**

✓ **Purpose:**

Remove duplicate values from result

✓ **Example:**

```
SELECT DISTINCT country FROM author;
```

◆ **6. Grouping Data (GROUP BY)**

✓ **Purpose:**

Group rows with same values into **summary results**

✓ **Syntax:**

```
SELECT column_name, COUNT(*)  
FROM table_name  
GROUP BY column_name;
```

✓ **Example:**

```
SELECT country, COUNT(*)  
FROM author  
GROUP BY country;
```

◆ **7. Naming Columns (AS)**

✓ **Purpose:**

Rename output columns for better readability

✓ **Example:**

```
SELECT country, COUNT(*) AS total_authors  
FROM author  
GROUP BY country;
```

◆ **8. Filtering Groups (HAVING)**

✓ **Purpose:**

Filter grouped data (used **with GROUP BY**)

✓ **Difference:**

- WHERE → filters rows
- HAVING → filters groups

✓ **Syntax:**

```
SELECT column_name, COUNT(*)  
FROM table_name  
GROUP BY column_name  
HAVING COUNT(*) > value;
```

✓ **Example:**

```
SELECT country, COUNT(*) AS total  
FROM author  
GROUP BY country  
HAVING total > 4;
```

 KEY TAKEAWAYS

- LIKE → pattern matching
- BETWEEN → range filtering
- IN → multiple values filtering
- ORDER BY → sorting results
- DISTINCT → remove duplicates
- GROUP BY → group data
- HAVING → filter grouped results

LAB: STRING PATTERNS, SORTING AND GROUPING

3.2 BUILT-IN SQL FUNCTIONS

◆ 1. What are Built-in Functions?

- Predefined functions available in databases
- Used to **perform operations directly in SQL queries**
- Help:
 - Reduce data transfer
 - Improve performance
 - Simplify queries

◆ 2. Aggregate Functions (Column Functions)

✓ Definition:

Operate on **multiple rows** and return **single value**

✓ Common Functions:

Function	Description
SUM()	Total of values
MIN()	Smallest value
MAX()	Largest value
AVG()	Average value
COUNT()	Number of rows

✓ Syntax:

```
SELECT FUNCTION(column_name) FROM table_name;
```

✓ Examples:

-- Total cost

```
SELECT SUM(COST) FROM PETRESCUE;
```

-- Maximum quantity

```
SELECT MAX(QUANTITY) FROM PETRESCUE;
```

-- Average cost

```
SELECT AVG(COST) FROM PETRESCUE;
```

✓ **Using Alias:**

```
SELECT SUM(COST) AS total_cost FROM PETRESCUE;
```

✓ **With Conditions:**

```
SELECT MIN(ID)
FROM PETRESCUE
WHERE ANIMAL = 'Dog';
```

✓ **With Calculations:**

```
SELECT AVG(COST / QUANTITY)
FROM PETRESCUE
WHERE ANIMAL = 'Dog';
```

◆ **3. Scalar Functions**

✓ **Definition:**

Operate on **single values**

◆ **(a) Numeric Functions**

✓ **Example:**

```
SELECT ROUND(COST) FROM PETRESCUE;
```

✓ **Common Functions:**

Function	Description
LENGTH()	Length of string
UPPER() / UCASE()	Convert to uppercase
LOWER() / LCASE()	Convert to lowercase

✓ Examples:

-- Length of animal names

```
SELECT LENGTH(ANIMAL) FROM PETRESCUE;
```

-- Convert to uppercase

```
SELECT UPPER(ANIMAL) FROM PETRESCUE;
```

-- Convert to lowercase

```
SELECT LOWER(ANIMAL) FROM PETRESCUE;
```

✓ Using in WHERE:

```
SELECT *  
FROM PETRESCUE  
WHERE LOWER(ANIMAL) = 'cat';
```

✓ Combining Functions:

```
SELECT DISTINCT UPPER(ANIMAL)  
FROM PETRESCUE;
```

◆ 4. Date and Time Functions

✓ SQL Date Types:

- DATE → YYYY-MM-DD
- TIME → HH:MM:SS
- TIMESTAMP → Date + Time

◆ Extract Functions:

Function	Purpose
DAY()	Extract day
MONTH()	Extract month
YEAR()	Extract year
HOUR()	Extract hour

✓ **Example:**

```
SELECT DAY(RESCUE_DATE)
FROM PETRESCUE
WHERE ANIMAL = 'Cat';
```

◆ **Using in WHERE:**

```
SELECT COUNT(*)
FROM PETRESCUE
WHERE MONTH(RESCUE_DATE) = 5;
```

◆ **Date Arithmetic**

✓ **Add days:**

```
SELECT DATE_ADD(RESCUE_DATE, INTERVAL 3 DAY)
FROM PETRESCUE;
```

✓ **Difference between dates:**

```
SELECT CURRENT_DATE - RESCUE_DATE
FROM PETRESCUE;
```

◆ **Current Date/Time:**

```
SELECT CURRENT_DATE;
SELECT CURRENT_TIME;
```

 KEY TAKEAWAYS

- **Aggregate functions** → work on multiple rows (SUM, AVG, etc.)
- **Scalar functions** → work on single values (ROUND, LENGTH, etc.)
- **String functions** → manipulate text (UPPER, LOWER)
- **Date functions** → extract and manipulate dates
- Functions can be used in:
 - SELECT
 - WHERE
 - Combined with other functions

LAB: BUILT-IN FUNCTIONS

3.3 SUB-QUERIES AND NESTED SELECTS

◆ 1. What is a Subquery?

- A **subquery (nested query)** is a query inside another query
- Written inside **parentheses ()**
- Used to perform **complex operations** in a single SQL statement

◆ 2. Why Use Subqueries?

- To **overcome limitations** of aggregate functions
- To **break complex problems into smaller steps**
- To make queries **more powerful and flexible**

◆ 3. Subquery in WHERE Clause

✓ Use Case:

Filter data based on result of another query

✓ Example: Employees earning more than average salary

```
SELECT employee_id, first_name, last_name, salary
FROM employees
WHERE salary > (
    SELECT AVG(salary) FROM employees
);
```

✓ Key Point:

- Aggregate functions like AVG() **cannot always be directly used in WHERE**
- Subquery solves this problem

◆ 4. Subquery in SELECT Clause (Column Expression)

✓ Use Case:

Add calculated values as a column

✓ Example:

```
SELECT employee_id, salary,  
       (SELECT AVG(salary) FROM employees) AS avg_salary  
FROM employees;
```

✓ Benefit:

- Compare each row with overall value (e.g., avg salary)

◆ 5. Subquery in FROM Clause (Derived Table)

✓ Use Case:

Treat subquery result as a **temporary table**

✓ Example:

```
SELECT *  
FROM (  
    SELECT employee_id, first_name, last_name, department_id  
    FROM employees  
) AS employee_info;
```

✓ Key Concept:

- Called **Derived Table / Table Expression**
- Useful when:
 - Working with **multiple tables**
 - Performing **complex joins**
 - Hiding sensitive data

◆ 6. Types of Subqueries (Conceptual)

Type	Description
Single-row	Returns one value
Multi-row	Returns multiple values
Correlated	Depends on outer query (advanced)

◆ 7. Important Rules

- Subqueries are enclosed in ()
- Inner query executes **first**
- Result is passed to outer query
- Can be used in:
 - WHERE
 - SELECT
 - FROM

🔥 KEY TAKEAWAYS

- Subqueries help write **advanced SQL queries**
 - Useful for:
 - Comparisons (e.g., salary vs avg)
 - Filtering data dynamically
 - Creating temporary datasets
 - Types of placement:
 - WHERE → filtering
 - SELECT → calculated column
 - FROM → derived table
-

LAB: SUB-QUERIES AND NESTED SELECTS

3.4 WORKING WITH MULTIPLE TABLES

◆ 1. Ways to Access Multiple Tables

You can retrieve data from more than one table using:

1. **Subqueries**
2. **Implicit Joins**
3. **Explicit Joins** (INNER JOIN, OUTER JOIN – covered later)

◆ 2. Using Subqueries with Multiple Tables

✓ Concept

- A query inside another query
- Inner query provides data to outer query

◆ Example 1: Employees with valid Department IDs

```
SELECT *  
FROM employees  
WHERE dep_id IN (  
    SELECT dep_id  
    FROM departments  
);
```

✓ Outer query → Employees

✓ Subquery → Departments (filters results)

◆ Example 2: Employees from a specific location

```
SELECT *  
FROM employees  
WHERE dep_id IN (  
    SELECT dep_id  
    FROM departments  
    WHERE loc_id = 'L0002'  
);
```


- ✓ Location exists in Departments table
- ✓ Used to filter Employees

◆ **Example 3: Departments with high-salary employees**

```
SELECT dep_id, dep_name
FROM departments
WHERE dep_id IN (
    SELECT dep_id
    FROM employees
    WHERE salary > 70000
);
```

- ✓ Step 1: Find employees with salary > 70000
- ✓ Step 2: Get their department details

◆ **3. Implicit Joins**

✓ **Concept**

- Multiple tables listed in FROM clause
- No explicit JOIN keyword used

◆ **Example: Cartesian Product**

```
SELECT *
FROM employees, departments;
```

△ Produces all combinations of rows

→ Result size = rows of table1 × rows of table2

◆ **4. Restricting Implicit Join Results**

✓ **Using WHERE clause to match rows**

```
SELECT *
FROM employees, departments
WHERE employees.dep_id = departments.dep_id;
```

- ✓ Filters only matching department records
- ✓ Avoids Cartesian product

◆ 5. Fully Qualified Column Names

- Used when columns have same names in different tables

```
employees.dep_id = departments.dep_id
```

- ✓ Prevents ambiguity

◆ 6. Using Table Aliases

✓ Concept

- Short names for tables
- Improves readability

◆ Example:

```
SELECT *  
FROM employees E, departments D  
WHERE E.dep_id = D.dep_id;
```

- ✓ E → Employees
- ✓ D → Departments

◆ 7. Selecting Specific Columns from Multiple Tables

◆ Example 1: Employee ID + Department Name

```
SELECT E.emp_id, D.dep_name  
FROM employees E, departments D  
WHERE E.dep_id = D.dep_id;
```

◆ Example 2: Using Aliases in SELECT

```
SELECT E.emp_id, D.dep_id  
FROM employees E, departments D  
WHERE E.dep_id = D.dep_id;
```

🔥 KEY CONCEPTS SUMMARY

- **Subqueries** → Filter data across tables
- **Implicit Join** → Multiple tables in FROM
- **Cartesian Product** → All combinations (avoid unless needed)
- **WHERE clause** → Used to match rows between tables
- **Aliases** → Shorten table names
- **Fully qualified columns** → Avoid confusion

⚡ Quick Comparison

Method	Use Case
Subquery	When filtering based on another table
Implicit Join	Simple multi-table queries
Explicit Join	Best practice (more readable, scalable)

LAB: WORKING WITH MULTIPLE TABLES

MODULE 4: ACCESSING DATABASES USING PYTHON

4.1 INTRODUCTION TO DATABASE ACCESS USING PYTHON

◆ 1. Why use Python with Databases?

- Easy to learn and use
- Rich ecosystem for data science:
 - NumPy → numerical computing
 - Pandas → data analysis
 - Matplotlib → visualization

- SciPy → scientific computing
- Cross-platform (runs everywhere)
- Supports relational databases

◆ 2. What is a Cloud / Database Connection Workflow?

Basic Flow:

1. Python program connects to DB
2. SQL query is sent to DB
3. DB processes query
4. Results returned to Python
5. Data analyzed using Python tools

◆ 3. What is an API?

Definition:

- API (Application Programming Interface) = set of functions to interact with systems

◆ SQL API

- Used to send SQL queries to DBMS
- Helps:
 - Execute queries
 - Fetch results
 - Handle errors

◆ 4. Popular Database APIs

Database	API
MySQL	MySQL C API
PostgreSQL	psycopg2
IBM DB2	ibm_db
SQL Server	dblib
Oracle	OCI
Java DB access	JDBC
Windows DB access	ODBC

◆ 5. Jupyter Notebook

✓ What is it?

- Open-source web-based environment
- Used for:
 - Writing code
 - Visualization
 - Data analysis

✓ Features:

- Supports 40+ languages
- Interactive output (charts, tables)
- Easy sharing (GitHub, email)
- Used widely in Data Science

◆ 6. Python DB-API (Database API)

✓ Definition:

- Standard API for connecting Python with databases
- Write **one program** → **works with multiple DBs**

✓ Advantages:

- Simple and consistent
- Portable across databases
- Reduces need to learn multiple APIs

◆ 7. DB-API Libraries

Database	Python Library
IBM DB2	ibm_db
MySQL	mysql.connector
PostgreSQL	psycopg2
MongoDB	pymongo

◆ 8. Core DB-API Concepts

◆ 1. Connection Object

- Used to connect to database
- Manages transactions

◆ 2. Cursor Object

- Used to execute SQL queries
- Fetch results from database

◆ 9. Important DB-API Methods

◆ Connection Methods

Method	Purpose
cursor()	Create cursor
commit()	Save changes
rollback()	Undo changes
close()	Close connection

◆ 10. What is a Cursor?

✓ Definition:

- Control structure to process query results
- Works like a file pointer

✓ Functions:

- Execute queries
- Fetch rows one by one
- Track current position in result set

◆ 11. Steps to Connect Python to Database

✓ Step-by-Step Workflow:

```
import db_module # Step 1: Import library
conn = db_module.connect( # Step 2: Connect
    database="DB_NAME",
    user="username",
    password="password")
cursor = conn.cursor() # Step 3: Create cursor
cursor.execute("SELECT * FROM table") # Step 4: Execute query
results = cursor.fetchall() # Step 5: Fetch data
conn.close() # Step 6: Close connection
```

◆ 12. Transactions in DB-API

- **commit()** → Save changes permanently
- **rollback()** → Undo changes if error occurs

◆ 13. Important Notes

- Always **close connection** after use
 - Multiple cursors from same connection:
 - Share data changes
 - Different connections:
 - May or may not be isolated
-

🔥 KEY TAKEAWAYS

- Python connects to databases using **DB-API**
- **Connection object** → manages DB connection
- **Cursor object** → executes queries
- APIs allow communication between Python and DB
- Jupyter notebooks are ideal for database + data analysis
- Always use:
 - **commit()** for saving
 - **close()** to release resources

⚡ QUICK SUMMARY

Concept	Meaning
API	Interface to interact with DB
DB-API	Python standard for DB access
Connection	Connect to database
Cursor	Run queries
Commit	Save changes
Rollback	Undo changes

LAB: CREATING TABLES, INSERTING AND QUERYING DATA

4.2 ACCESSING DATABASES WITH SQL MAGIC

◆ 1. What are Magic Commands?

✅ Definition:

- Special commands in **Jupyter Notebook**
- Not standard Python code
- Provide extra functionality for data analysis

◆ 2. Types of Magic Commands

◆ Line Magics (%)

- Work on **single line**
- Similar to terminal commands

Example:

```
%pwd  
%ls
```

◆ Cell Magics (%%)

- Work on **entire cell**
- Can execute multiple lines
- Can run code in other languages

Example:

```
%%timeit  
for i in range(1000):  
    pass
```

◆ 3. Common Line Magic Commands

Command	Purpose
%pwd	Show current directory
%ls	List files
%history	Show command history
%reset	Clear variables
%who	List variables
%whos	Detailed variable info
%matplotlib inline	Show plots in notebook
%timeit	Measure execution time

◆ 4. Difference: Line vs Cell Magic

Feature	Line Magic	Cell Magic
Prefix	%	%%
Scope	One line	Entire cell
Example	%timeit x=5	%%timeit (whole cell)

◆ 5. Examples of Cell Magics

◆ HTML

```
%%html  
<h1>Hello World</h1>
```

◆ JavaScript

```
%%javascript  
alert("Hello World");
```

◆ Bash

```
%%bash  
echo "Hello World"
```

◆ Write to File

```
%%writefile file.txt  
This is saved in a file
```

◆ 6. What is SQL Magic?

✅ Definition:

- Allows running **SQL queries directly in Jupyter Notebook**
- No need to switch between Python and SQL tools

◆ 7. Steps to Use SQL Magic

✓ Step 1: Install library

```
!pip install ipython-sql
```

✓ Step 2: Load extension

```
%load_ext sql
```

✓ Step 3: Connect to Database (SQLite example)

```
%sql sqlite:///HR.db
```

✓ Step 4: Running SQL Queries

◆ Line Magic Query

```
%sql SELECT * FROM employee;
```

◆ Cell Magic Query

```
%%sql  
SELECT *  
FROM employee;
```

◆ 8. Workflow of SQL Magic

1. Install SQL magic library
2. Load extension
3. Connect to database
4. Run SQL queries directly in notebook

◆ 9. Key Advantages

- Run SQL + Python in same environment
- Faster data analysis workflow
- No need for separate SQL tools
- Easy integration with Pandas

🔥 KEY TAKEAWAYS

- Magic commands enhance Jupyter functionality
- % → single line | %% → entire cell
- SQL Magic allows direct database querying
- Requires:
 - ipython-sql installation
 - %load_ext sql
 - Database connection

⚡ Quick Summary

Concept	Meaning
Magic Commands	Special Jupyter commands
Line Magic	% (single line)
Cell Magic	%% (full cell)
SQL Magic	Run SQL in notebook
%sql	Execute SQL query
%%sql	Execute multi-line SQL

TUTORIAL: ACCESSING DATABASES WITH SQL MAGIC

4.3 PERFORM EXPLORATORY DATA ANALYSIS (EDA) USING PYTHON

- ◆ 1. Tools used:
 - **Pandas** → Data handling & analysis
 - **SQLite3** → Database storage
 - **Seaborn** → Data visualization

◆ 2. Dataset Used

- Dataset: McDonald's Menu Nutritional Data
- Source: Kaggle
- Contains:
 - Food items
 - Nutritional values (fat, sodium, sugar, protein, etc.)

◆ 3. SQLite3 (Database)

- Lightweight, **serverless database**
- No setup required
- Works directly with Python

◆ 4. Data Loading Process

✓ Step 1: Read CSV file

```
import pandas as pd
data = pd.read_csv("./menu.csv")
```

✓ Step 2: Connect to Database

```
import sqlite3
conn = sqlite3.connect("mcdonalds.db")
```

✓ Step 3: Store Data in Database

```
data.to_sql("mcdonalds_nutrition", conn)
```

✓ Step 4: Retrieve Data

```
df = pd.read_sql("SELECT * FROM mcdonalds_nutrition", conn)
```

◆ 5. Data Exploration (EDA)

✓ View Data

```
df.head()
```

✓ Summary Statistics

```
df.describe(include = 'all')
```

Insights:

- Total records: **260 food items**
- Categories: **9 unique types**
- Statistics available:
 - Mean
 - Median
 - Standard deviation
 - Min / Max

◆ 6. Sodium Analysis

⚠ Health Insight:

- Recommended sodium intake: **< 2000 mg/day**
- Excess sodium → high BP, swelling, health issues

✓ Find Maximum Sodium

```
df["Sodium"].describe()
```

- Max value $\approx$ **3600 mg**

✓ Find Index of Max Value

```
index = df["Sodium"].idxmax()
```

✓ Get Food Item Name

```
df.at[index, "Item"]
```

👉 Result: **Chicken McNuggets (40 pieces)**

◆ 7. Data Visualization

✓ Swarm Plot (Categorical Scatter Plot)

```
import matplotlib.pyplot as plt
%matplotlib inline
import seaborn as sns

### Categorical scatterplots

plot = sns.swarmplot(x="Category", y='Sodium', data=df)
plt.setp(plot.get_xticklabels(), rotation=70)
plt.title('Sodium Content')
plt.show()
```

✦ Purpose:

- Shows distribution of sodium across categories
- Helps identify **extreme values (outliers)**

✓ Scatter Plot (Joint Plot)

```
import matplotlib.pyplot as plt
%matplotlib inline
import seaborn as sns

plot = sns.jointplot(x="Protein", y='Total Fat', data=df)
plot.show()
```

✦ Purpose:

- Shows relationship between two variables

📊 Insight:

- **Positive correlation (~0.81)**
- Higher protein → higher fat

✦ Extra:

- Includes histograms
- Helps detect **outliers**

✓ Box Plot

```
import matplotlib.pyplot as plt
%matplotlib inline
import seaborn as sns

plot = sns.set_style("whitegrid")
ax = sns.boxplot(x=df["Sugars"])
plot.show()
```

✦ Purpose:

- Shows distribution of data

📊 Insights:

- Median sugar $\approx$ **30g**
- Outliers up to **128g sugar**

◆ 8. Important Concepts

✓ Correlation

- Range: **-1 to +1**
 - +1 $\rightarrow$ strong positive
 - -1 $\rightarrow$ strong negative
- Indicates relationship between variables

✓ Outliers

- Data points far from normal range
- Can indicate:

- Errors
- Rare events
- Important insights

✓ Box Plot Components

- Box → middle 50% data
- Line → median
- Points → outliers

◆ 9. Key Functions Summary

Function	Purpose
read_csv()	Load CSV data
connect()	Connect to DB
to_sql()	Store data in DB
read_sql()	Retrieve data
describe()	Summary stats
idxmax()	Index of max value
at[]	Access specific value

🔥 FINAL SUMMARY

- Use **Pandas** + **SQLite** for data storage & analysis
- Perform **EDA** using:
 - head() → preview data
 - describe() → statistics
- Use **Seaborn** for visualization:

- Swarm plot → distribution
- Scatter plot → relationships
- Box plot → spread & outliers
- Identify:
 - Patterns
 - Correlations
 - Extreme values

LAB: ANALYZING A REAL-WORLD DATA SET

MODULE 5: ASSIGNMENT PREPARATION

5.1 WORKING WITH REAL-WORLD DATASETS

◆ 1. What are Real-World Datasets?

- Usually available as **CSV files**
- CSV = **Comma-Separated Values**
- Text files with:
 - Rows → records
 - Columns → attributes

📌 Example: DOGS.csv

ID	Dog Name	Breed
1	Wolfie	German Shepherd
2	Fluffy	Pomeranian
3	Huggy	Labrador

✓ Key Points:

- First row = **Column names (Header)**

- Remaining rows = **Data values**

◆ 2. Loading CSV into Database

✓ Using phpMyAdmin

Steps:

1. Select database
2. Click **Import**
3. Choose CSV file
4. Select format = CSV
5. ☒ Tick: *First row contains column names*
6. Click **Go**

✚ Notes:

- Table name = usually **file name (lowercase)**
- DB automatically:
 - Creates table
 - Inserts data

◆ 3. Working with Column Names

⚠ Problem:

Column names may contain:

- Spaces
- Special characters

✓ Solution:

Use **backticks (`)**

```
SELECT `Dog Name` FROM dogs;
```

✗ Wrong:

- 'Dog Name'
- "Dog Name"

◆ 4. Writing Long Queries

✓ Best Practice:

- Split queries into multiple lines

✓ In Python Notebook:

Use \ for continuation

```
query = "SELECT * FROM table \  
WHERE condition"
```

✓ With SQL Magic (%%sql):

- No need for backslash
- Entire cell treated as SQL

◆ 5. Using SQL in Python (Pandas)

✓ Query using read_sql

```
query = 'SELECT * FROM table'  
df = pd.read_sql(query, conn)
```

✓ Handling Quotes

- Use **single quotes** for Python string
- Use **double quotes** for column names

```
query = 'SELECT "Dog Name" FROM dogs'
```

✓ Escape single quotes

```
query = 'SELECT * FROM dogs WHERE name = \'Fluffy\''
```

◆ 6. Limiting Data (IMPORTANT)

✗ Problem:

- Large tables = slow queries

✓ Solution: LIMIT

```
SELECT * FROM census_data  
LIMIT 3;
```

✦ Benefit:

- Faster execution
- Preview data easily

◆ 7. Getting Table Information

✦ Why Needed?

- Don't remember table names
- Need schema details

◆ 8. Get List of Tables

✓ SQLite

```
SELECT name FROM sqlite_master WHERE type='table';
```

✓ MySQL

```
SHOW TABLES;
```

◆ 9. Get Column Details

✓ SQLite

```
PRAGMA table_info(table_name);
```

✓ MySQL

DESCRIBE table_name;

FINAL SUMMARY (QUICK REVISION)

- Real-world data → mostly **CSV files**
 - First row → **column names**
 - Use **phpMyAdmin** to import data
 - Use **backticks (`)** for column names with spaces
 - Use **LIMIT** to fetch small data samples
 - Use **read_sql()** to query database in Python
 - Use system tables to:
 - View tables
 - View columns
-

LAB: WORKING WITH A REAL WORLD DATA-SET USING SQL AND PYTHON

FINAL ASSIGNMENT: DATABASE QUERYING USING SQLITE

MODULE 6: ADVANCED SQL FOR DATA ENGINEERS

6.1 VIEWS, STORED PROCEDURES, AND TRANSACTIONS

1. Views in SQL

What is a View?

- A **View** is a virtual table created from one or more tables or views.
- It does not store data itself.
- It stores only the SQL query definition.

- Data is retrieved dynamically from the base tables whenever the view is queried.

Definition

A view is an alternative way of representing data that exists in one or more tables.

2. Features of Views

- Can contain:
 - All columns or selected columns
 - Data from one or multiple tables
- Behaves like a normal table
- Can be queried using SELECT
- Supports:
 - INSERT
 - UPDATE
 - DELETE(changes affect the original/base table)

3. Why Use Views?

a) Hide Sensitive Data

Views can restrict access to confidential information.

Example:

Instead of showing:

- Salary
- Birth date
- Tax details

A view may show only:

- Employee ID
- Name

- Department ID

b) Simplify Complex Queries

Views can combine multiple tables and simplify repeated queries.

c) Provide Restricted Access

Users can access the view without accessing the original table.

d) Show Relevant Data Only

Views help display only the information needed for a particular process or user.

4. Syntax to Create a View

```
CREATE VIEW <view name> (<column_alias_1>,  
<column_alias_2>, ... <column_alias_n>)  
    AS SELECT <column_1> , <column_2>, ... <column_n>  
    FROM <table name>  
    WHERE <predicate>;
```

5. Example: Creating a View

Create EMPINFO View

```
CREATE VIEW EMPINFO (EMP_ID, FIRSTNAME, LASTNAME, ADDRESS,  
JOB_ID, MANAGER_ID, DEP_ID)  
    AS SELECT EMP_ID, F_NAME, L_NAME, ADDRESS, JOB_ID,  
MANAGER_ID, DEP_ID  
    FROM EMPLOYEES;
```

Purpose

- Shows non-sensitive employee information
- Hides salary and birthdate

6. Using WHERE Clause in Views

You can filter rows while creating a view.

Example

```
CREATE VIEW EMPINFO (EMP_ID, FIRSTNAME, LASTNAME, ADDRESS,  
JOB_ID, MANAGER_ID, DEP_ID)  
AS SELECT EMP_ID, F_NAME, L_NAME, ADDRESS, JOB_ID,  
MANAGER_ID, DEP_ID  
FROM EMPLOYEES  
WHERE MANAGER_ID = '30002'
```

Result

- Only employees managed by 30002 appear in the view.

7. Querying a View

Views are queried just like tables.

Example

```
SELECT * FROM EMPINFO;
```

8. Important Characteristics of Views

Views are Dynamic

- Views always display the latest data from the base table.
- If base table data changes, the view reflects the changes automatically.

Views Store Definition Only

- The SQL statement is stored.
- Actual data remains in the original tables.

9. Clauses Allowed in Views

The SELECT statement used to create views may include:

- WHERE
- GROUP BY
- HAVING

Not Allowed

- ORDER BY
- Host variables

10. Dropping a View

To remove a view permanently:

```
DROP VIEW view_name;
```

Example

```
DROP VIEW EMPINFO;
```

11. Advantages of Views

Advantage	Description
Security	Hide sensitive columns
Simplicity	Simplify complex queries
Reusability	Reuse common queries
Data Abstraction	Users don't need table structure knowledge
Restricted Access	Provide controlled data access

KEY POINTS SUMMARY

- A view is a virtual table based on SQL queries.
- Views do not store actual data.
- Views can simplify data access and improve security.
- Use CREATE VIEW to create views.
- Views can include filtering using WHERE.
- Views behave like regular tables for querying.
- Use DROP VIEW to delete a view.
- Changes in base tables are automatically reflected in views.

IMPORTANT SQL COMMANDS

Create View

```
CREATE VIEW view_name AS  
SELECT columns  
FROM table_name;
```

Query View

```
SELECT * FROM view_name;
```

Delete View

```
DROP VIEW view_name;
```

LAB- USING VIEWS

6.2 STORED PROCEDURES

1. What is a Stored Procedure?

A **Stored Procedure** is a collection of SQL statements stored and executed on the database server.

- Instead of sending multiple SQL statements from client to server, all statements are grouped into one procedure.
- The client only sends one command to execute the procedure.

2. Key Features of Stored Procedures

- Stored inside the database
- Executed on the server side
- Can accept input parameters
- Can return output values/results
- Can perform:
 - CREATE
 - READ
 - UPDATE
 - DELETE (CRUD)
- Can include:
 - Conditional statements
 - Loops
 - Variables

3. Languages Used for Stored Procedures

Stored procedures can be written using:

- SQL
- PL/SQL
- Java
- C
- Other supported languages

Example:

- IBM DB2 supports SQL PL, Java, C, etc.

4. Benefits of Stored Procedures

a) Reduced Network Traffic

- Multiple SQL statements are executed with a single procedure call.
- Fewer requests are sent between client and server.

b) Better Performance

- Processing occurs directly on the database server.
- Only the final result is returned to the client.

c) Code Reusability

- Multiple applications can use the same stored procedure.

d) Improved Security

- Developers do not need direct access to tables/columns.
- Server-side validation can be implemented before storing data.

5. Important Note

SQL is not a full programming language.

- Avoid putting all business logic inside stored procedures.
- Use them mainly for database operations and validations.

6. Syntax to Create a Stored Procedure

```
DELIMITER $$  
CREATE PROCEDURE procedure_name(parameters)  
BEGIN  
    SQL statements;  
END $$  
DELIMITER ;
```

7. Example: Stored Procedure to Update Salary

Scenario

- Increase salary based on employee rating:
 - Rating = 1 → 10% raise
 - Others → 5% raise

SQL Code

```
DELIMITER $$
CREATE PROCEDURE UPDATE_SAL (IN empNum CHAR(6), IN rating
SMALLINT)
BEGIN
UPDATE employees SET salary = salary * 1.10 WHERE emp_id
= empNum AND rating = 1;
UPDATE employees SET salary = salary * 1.05 WHERE emp_id
= empNum AND rating <> 1;
END
$$
DELIMITER ;
```

8. Understanding the Example

Part	Description
CREATE PROCEDURE	Creates procedure
IN	Input parameter
BEGIN ... END	Procedure block
IF ... ELSE	Conditional logic
UPDATE	Updates employee salary
DELIMITER \$\$	Changes statement terminator

9. Why Change the Delimiter?

Normally SQL statements end with ;

A stored procedure contains multiple statements.

So delimiter is temporarily changed to:

\$\$

After procedure creation, delimiter is reset back to:

;

10. Calling a Stored Procedure

Use the CALL statement.

Syntax

```
CALL procedure_name(parameters);
```

Example

```
CALL UPDATE_SAL(1001, 1)
```

Meaning

- Employee ID = 1001
- Rating = 1
- Salary increases by 10%

11. Advantages of Stored Procedures

Advantage	Explanation
Performance	Faster execution on server
Security	Restricts direct table access
Reusability	Same logic reused everywhere
Scalability	Efficient for large systems

Advantage	Explanation
Data Integrity	Validation can be enforced

KEY POINTS SUMMARY

- Stored procedures are SQL programs stored in the database.
- They execute on the database server.
- Procedures reduce network traffic and improve performance.
- Use CREATE PROCEDURE to create procedures.
- Use parameters to pass input values.
- Use CALL to execute a procedure.
- Stored procedures support conditional logic and CRUD operations.
- Delimiters are changed temporarily while defining procedures.

IMPORTANT SQL COMMANDS

Create Procedure

```
CREATE PROCEDURE procedure_name()
BEGIN
    SQL statements;
END;
```

Call Procedure

```
CALL procedure_name();
```

Example Procedure Call

```
CALL UPDATE_SAL(1001,1);
```

LAB: STORED PROCEDURES

6.3 ACID TRANSACTIONS

1. What is a Transaction?

A **Transaction** is a single logical unit of work in a database.

- It may contain:
 - One SQL statement
 - Multiple SQL statements
- A transaction is successful only if all operations complete successfully.

2. Important Rule of Transactions

A transaction follows the principle:

Either ALL operations succeed OR NONE succeed.

This ensures the database always remains correct and stable.

3. Real-Life Example of a Transaction

Example: Online Purchase

Suppose Rose buys boots worth \$200.

The following actions must happen together:

1. Deduct \$200 from Rose's account
2. Add \$200 to Shoe Shop's account
3. Reduce shoe inventory by 1

SQL Operations

```
UPDATE customer_account  
SET balance = balance - 200  
WHERE customer_name = 'Rose';
```

```
UPDATE shop_account  
SET balance = balance + 200  
WHERE shop_name = 'Shoe Shop';
```

```
UPDATE inventory
SET stock = stock - 1
WHERE product = 'Boots';
```

4. What Happens if One Statement Fails?

Suppose:

- Rose does not have enough money.

Then:

- Money should NOT be added to the shop.
- Inventory should NOT decrease.

All changes must be undone.

This is why transactions are important.

5. ACID Transactions

ACID transactions guarantee reliability in databases.

Meaning of ACID

Letter	Meaning	Explanation
A	Atomicity	All operations succeed or none
C	Consistency	Database remains valid before and after transaction
I	Isolation	Other processes cannot interfere during execution
D	Durability	Changes remain permanently saved

Atomicity

Definition

A transaction is treated as one indivisible unit.

Example

If one UPDATE fails:

- All previous updates are cancelled.

Consistency

Definition

The database must remain correct and valid.

Example

Money cannot disappear or be created accidentally.

Isolation

Definition

Transactions do not interfere with each other.

Example

Two users cannot modify the same record simultaneously in conflicting ways.

Durability

Definition

Once committed, changes remain permanently saved.

Example

Even after:

- Power failure
- System crash

Committed data remains stored.

6. Transaction Control Commands (TCL)

Command	Purpose
BEGIN	Starts transaction
COMMIT	Saves changes permanently
ROLLBACK	Undoes changes

BEGIN Transaction

Starts a transaction.

BEGIN;

Note

In some databases like DB2 Cloud:

- BEGIN is implicit (automatic)

COMMIT Command

Saves all successful changes permanently.

Example

COMMIT;

Use When

- All SQL statements execute successfully.

ROLLBACK Command

Cancels all changes made during the transaction.

Example

ROLLBACK;

Use When

- Any statement fails
- Errors occur

7. Example of Full Transaction

BEGIN

```
UPDATE BankAccounts  
SET Balance = Balance - 200  
WHERE AccountName = 'Rose';
```

```
UPDATE BankAccounts  
SET Balance = Balance + 200  
WHERE AccountName = 'Shoe Shop';
```

```
UPDATE ShoeShop  
SET Stock = Stock - 1  
WHERE Product = 'Boots';
```

COMMIT

8. Example with ROLLBACK

BEGIN

```
UPDATE BankAccounts  
SET Balance = Balance - 200  
WHERE AccountName = 'Rose';
```

```
UPDATE BankAccounts  
SET Balance = Balance + 200  
WHERE AccountName = 'Shoe Shop';
```

```
UPDATE ShoeShop  
SET Stock = Stock - 1  
WHERE Product = 'Boots';
```

ROLLBACK

Result

- All previous updates are undone.

9. Transactions in Programming Languages

SQL transactions can be used inside:

- Python
- Java
- C
- R

Using APIs like:

- JDBC (Java)
- ibm_db (Python)
- MySQL connectors

10. Example in Application Code

```
void main ()
{
    EXEC SQL UPDATE BankAccounts
        SET Balance = Balance - 200
        WHERE AccountName = 'Rose';
    EXEC SQL UPDATE BankAccounts
        SET Balance = Balance + 200
        WHERE AccountName = 'Shoe Shop';
    EXEC SQL UPDATE ShoeShop
        SET Stock = Stock - 1
        WHERE Product = 'Boots';
    FINISHED:
    EXEC SQL COMMIT WORK;
    return;

    SQLERR:
    EXEC SQL WHENEVER SQLERROR CONTINUE;
    EXEC SQL ROLLBACK WORK;
    return;
}
```

Applications can:

- Detect errors
- Decide whether to commit or rollback

11. Advantages of ACID Transactions

Advantage	Explanation
Data Integrity	Keeps data accurate
Reliability	Prevents incomplete updates
Consistency	Maintains stable database state
Error Recovery	Rollback restores previous state

Advantage	Explanation
Multi-user Safety	Prevents conflicts

KEY POINTS SUMMARY

- A transaction is a logical unit of work.
- Transactions may contain multiple SQL statements.
- ACID ensures reliable database operations.
- ACID stands for:
 - Atomicity
 - Consistency
 - Isolation
 - Durability
- Use:
 - BEGIN → start transaction
 - COMMIT → save changes
 - ROLLBACK → undo changes
- Transactions are widely used in banking, shopping, and reservation systems.

LAB: COMMITTING AND ROLLING BACK A TRANSACTION

6.3 SQL JOIN STATEMENTS

1. JOIN Overview

What is a JOIN?

- A **JOIN** is an SQL operator used to combine rows from two or more tables.
- Tables are joined based on a relationship between columns in those tables.

Definition

JOIN combines rows from two or more tables using related columns.

2. Why are JOINS Needed?

- A single table may not contain all required information.
- Real-world databases store data in multiple related tables.
- JOINS help retrieve related data from different tables in one query.

Example

Library database:

- Borrower table → borrower details
- Loan table → loan details
- Copy table → copy/book details

To know:

Which borrower has which book copy on loan

You must JOIN these tables.

3. Primary Key and Foreign Key

Primary Key (PK)

- Uniquely identifies each row in a table.
- Cannot contain duplicate values.

Example

Borrower_ID

Book_ID

Author_ID

Foreign Key (FK)

- A column that refers to the Primary Key of another table.
- Creates a relationship between tables.

Example

Loan.Borrower_ID

references

Borrower.Borrower_ID

4. Types of JOINS

SQL JOINS are mainly divided into:

1. INNER JOIN
2. OUTER JOIN
 - LEFT JOIN
 - RIGHT JOIN
 - FULL OUTER JOIN

INNER JOIN

Definition

- Returns only rows that have matching values in both tables.

When to Use

- When you only need matching records from both tables.

INNER JOIN Syntax

```
SELECT columns  
FROM table1  
INNER JOIN table2  
ON table1.column = table2.column;
```

Example

```
SELECT B.BORROWER_ID, B.LASTNAME, B.COUNTRY,  
        L.BORROWER_ID, L.LOAN_DATE  
FROM BORROWER B INNER JOIN LOAN L  
  
        ON B.BORROWER_ID = L.BORROWER_ID
```

Explanation

- B and L are aliases.
- Only matching Borrower_ID values appear.
- Non-matching rows are excluded.

Key Points

- Most commonly used JOIN.
- Uses matching values only.
- Non-matching rows are not displayed.

OUTER JOINS

Definition

- Return matching rows plus non-matching rows from one or both tables.

LEFT OUTER JOIN (LEFT JOIN)

Definition

- Returns:
 - All rows from the LEFT table
 - Matching rows from the RIGHT table
- If no match exists, NULL values appear.

LEFT JOIN Syntax

```
SELECT columns  
FROM table1  
LEFT JOIN table2  
ON table1.column = table2.column;
```

Example

```
SELECT B.BORROWER_ID, B.LASTNAME, B.COUNTRY,  
        L.BORROWER_ID, L.LOAN_DATE  
FROM BORROWER B LEFT JOIN LOAN L  
        ON B.BORROWER_ID = L.BORROWER_ID
```

Result

- All borrowers appear.
- Borrowers without loans show NULL in Loan_Date.

RIGHT OUTER JOIN (RIGHT JOIN)

Definition

- Returns:
 - All rows from the RIGHT table
 - Matching rows from the LEFT table
- Non-matching rows from the left table become NULL.

RIGHT JOIN Syntax

```
SELECT columns  
FROM table1  
RIGHT JOIN table2  
ON table1.column = table2.column;
```

Example

```
SELECT B. BORROWER_ID, B.LASTNAME, B.COUNTRY,  
       L. BORROWER_ID, L.LOAN_DATE  
FROM BORROWER B RIGHT JOIN LOAN L  
       ON B.BORROWER_ID = L.BORROWER_ID
```

Result

- All loan records appear.
- If a borrower record is missing, borrower columns show NULL.

FULL OUTER JOIN

Definition

- Returns:
 - All matching rows
 - All non-matching rows from both tables

FULL JOIN Syntax

```
SELECT columns  
FROM table1  
FULL OUTER JOIN table2  
ON table1.column = table2.column;
```

Example

```
SELECT B.BORROWER_ID, B.LASTNAME, B.COUNTRY,  
       L.BORROWER_ID, L.LOAN_DATE  
FROM BORROWER B FULL JOIN LOAN L  
       ON B.BORROWER_ID = L.BORROWER_ID
```

Result

- Includes:
 - Matching rows
 - Borrowers without loans
 - Loans without borrower records

5. Aliases in JOINS

What are Aliases?

- Short names for tables.

Example

Borrower B
Loan L

Benefits

- Makes queries shorter
- Improves readability
- Useful in large queries

6. Multiple Table JOINS

Example

To find:

- Borrower details
- Loan details
- Copy details

You can JOIN multiple tables.

```
SELECT *  
FROM Borrower B  
INNER JOIN Loan L  
ON B.Borrower_ID = L.Borrower_ID  
INNER JOIN Copy C  
ON L.Copy_ID = C.Copy_ID;
```

7. Important Concepts

JOIN Predicate

- Condition specified in the ON clause.
- Defines how tables are related.

Example

ON B.Borrower_ID = L.Borrower_ID

KEY TAKEAWAYS

- JOIN combines rows from multiple tables.
- Tables are linked using:
 - Primary Keys
 - Foreign Keys
- INNER JOIN returns matching rows only.
- OUTER JOINS also return non-matching rows.

- Types of OUTER JOINS:
 - LEFT JOIN
 - RIGHT JOIN
 - FULL OUTER JOIN
- Aliases improve readability.
- JOINS are essential in relational databases.

LAB: JOINS
